

SQL Injection: Mechanism, Impact, Detection, and Prevention

1. Introduction

SQL Injection (SQLi) is a server side attack in which an attacker inserts malicious SQL syntax into input that is improperly incorporated into a database query. If an application fails to separate user input from SQL commands, the attacker may alter the intended query and gain unauthorized access to database information or perform unauthorized database operations depending on the privileges. SQL Injection commonly affects database-driven applications, including web applications and APIs, but it is not limited to websites or login pages.

2. Technical Overview

SQL Injection targets vulnerable applications that use SQL to communicate with a back-end database. The main prerequisite is an application that places untrusted user input directly into a dynamically constructed SQL query, typically through unsafe string concatenation. **Merely using SQL does not make an application vulnerable.** The vulnerability exists when attacker-controlled input can change the structure or meaning of the SQL statement.

The potential impact depends on the application's database privileges and the vulnerability itself. A successful attack may allow unauthorized reading, modification, or deletion of database records and, in some cases, further compromise of the underlying system.

3. How the Attack Works

Consider a vulnerable login query:

```
SELECT * FROM users WHERE username = 'admin' AND password = 'password123'
```

The application may insert the values entered by the user directly into this query. An attacker can attempt to alter its logic by supplying specially crafted input.

For example, an attacker may use an **OR** condition that is always true:

```
SELECT * FROM users WHERE username = 'admin' OR '1'=1' AND password = 'password123'
```

Because **AND** has higher precedence than **OR**, this expression is evaluated as:

```
username = 'admin' OR ('1'=1' AND password = 'password123')
```

A more reliable authentication-bypass technique uses a comment to remove the remaining password condition. For example:

```
SELECT * FROM users WHERE username = 'admin'--' AND password = 'password'
```

Here, the comment marker causes the remaining part of the statement to be treated as a comment. If the application is vulnerable and the database accepts the syntax, the password condition may therefore be ignored. The exact comment syntax varies between database systems.

SQL Injection is not limited to authentication bypass. Depending on the application's query and database permissions, attackers may attempt to retrieve, modify, or delete data.

4. MITRE ATT&CK Mapping

SQL Injection can be associated with **MITRE ATT&CK T1190 – Exploit Public-Facing Application** when an attacker uses an SQL Injection vulnerability in an internet-facing application to gain initial access. T1190 itself is a broader technique covering exploitation of weaknesses in public-facing applications and services; it is not an SQL Injection-specific technique.

5. Real-World Examples

Sony PlayStation Network (2011): Sony experienced a major breach in 2011 that affected approximately 77 million accounts. SQL Injection was also used in a separate 2011 attack against Sony by LulzSec, which exposed information belonging to more than one million customers. Because the exact attack path of the major PSN breach has been disputed, it is more accurate not to attribute the entire 77-million-account incident solely to SQL Injection.

7-Eleven and other retailers (2007–2008): SQL Injection techniques were used in a wider criminal campaign associated with Albert Gonzalez and his co-conspirators. The victims included 7-Eleven, Heartland Payment Systems, and Hannaford Brothers. The overall campaign involved the theft of more than 130 million credit and debit card numbers; this figure should not be attributed to 7-Eleven alone.

6. Detection

Organizations should monitor application and database logs for unusual SQL errors, repeated abnormal requests, unexpected database queries, authentication anomalies, and suspicious changes to database records. Web application firewalls (WAFs) and security monitoring systems can provide additional visibility, but they should not replace secure application development.

7. Prevention and Mitigation

The primary defense against SQL Injection is the use of **prepared statements with parameterized queries**. These mechanisms separate SQL code from user-supplied data, preventing input from being interpreted as part of the SQL command.

Additional controls include:

- **Server-side input validation:** Reject unexpected or invalid input where appropriate.
- **Least privilege:** Give application database accounts only the permissions they require.
- **Secure stored procedures or ORM frameworks:** Use them safely and avoid dynamic SQL construction.
- **Web Application Firewall (WAF):** Detect and block some malicious requests as an additional security layer.
- **Regular logging and monitoring:** Identify suspicious requests and database activity.
- **Security testing:** Perform code reviews and vulnerability testing to identify unsafe query construction.

Input validation and WAFs are supporting controls; they should not be treated as substitutes for parameterized queries.

8. Conclusion

SQL Injection occurs when an application improperly combines untrusted input with SQL commands. Its impact can range from authentication bypass to unauthorized access, modification, or deletion of database information. The most effective prevention is to use parameterized queries and maintain strong access controls. Secure coding, input validation, monitoring, and vulnerability testing provide additional layers of protection.

References

1. MITRE ATT&CK. *T1190: Exploit Public-Facing Application*.
2. BBC News. "Sony PlayStation Network hit by cyber attack." BBC News, 2011. [BBC News article](#)
3. U.S. Department of Justice. "Alleged International Hacker Indicted for Massive Attack on U.S. Retail and Banking Networks." August 17, 2009.

